



Politechnika Łódzka

Miszczak Paweł (246967)

Projekt i realizacja regulatora prędkości obrotowej silnika prądu stałego (DC) z wykorzystaniem modulacji szerokości impulsów (PWM).

Niniejsza praca jest wynikiem mojej samodzielnej pracy. Świadomy odpowiedzialności przed władzami uczelni, prawem i Bogiem, oświadczam, że jest to praca wykonana samodzielnie i własnoręcznie.

Spis treści:

1. Wstęp.....	4
2. Zasada działania - ogólny zarys.....	5
3. Stos technologiczny.....	8
4. Opis kodu.....	9
4.1. constants.py - parametry algorytmu.....	9
4.1.1. Model i geometria tarczy.....	10
4.1.2. OCR.....	10
4.1.3. Przycinanie.....	10
4.1.4. Maskowanie wskazówek.....	11
4.1.5. Transformacja Hougha.....	12
4.1.6. Deduplikacja kandydatów.....	12
4.1.7. Pomiar promieniem.....	13
4.1.8. Wyznaczanie godziny.....	13
4.1.9. Kolory i rysowanie wizualizacji.....	13
4.1.10. Serwer.....	14
4.2. app.py - warstwa aplikacyjna.....	15
4.3. clock_reader.py - algorytm analizy obrazu.....	17
4.3.1. crop_dial() - wycięcie i powiększenie tarczy.....	17
4.3.2. ocr_digits() - rozpoznawanie cyfr.....	18
4.3.3. line_intersection() - obliczenie przecięcia prostych.....	19
4.3.4. hands_mask() - segmentacja wskazówek.....	21
4.3.5. hough_candidates() - wykrywanie kandydatów na wskazówki.....	23
4.3.6. dedup_hands() - usuwanie powtarzających się detekcji.....	25
4.3.7. dial_otsu() - określenie charakteru wskazówki względem tła.....	26
4.3.8. ray_darkness_length() - niezależny pomiar długości wskazówki.....	26
4.3.9. clock_time() - przeliczenie geometrii na godzinę.....	28
4.3.10. draw_overlay() - wizualizacja procesu.....	29
4.4. read_clock() - główny potok przetwarzania.....	30
4.5. Przepływ danych w programie.....	31
5. Uczenie modelu YOLO.....	32
5.1. Przygotowanie zbioru danych.....	32
5.2. Ręczna anotacja obrazów.....	33
5.3. Proces treningu.....	33

6. Wdrożenie produkcyjne.....	34
6.1. Automatyzacja instalacji środowiska.....	35
6.2. Uruchamianie aplikacji za pomocą Gunicorn i Supervisor.....	35
6.3. Konfiguracja Nginx.....	36
6.4. Konfiguracja HTTPS.....	37
6.5. Dobór zasobów serwera.....	37
6.6. Ograniczenia konfiguracji produkcyjnej.....	38
6.7. Pasek postępu w interfejsie użytkownika.....	38
6.8. Ograniczona równoległość przetwarzania.....	39
6.9. Zależność działania od jakości zdjęcia.....	40
6.10. Zakres dalszej rozbudowy.....	40
7. Podsumowanie.....	40

1. Wstęp

Projekt został zrealizowany w ramach przedmiotu Systemy Wizyjne (02 67 5361 01). Przedmiotem zadania było opracowanie systemu wizyjnego przeznaczonego do rozpoznawania czasu na analogowym zegarze wskazówkowym w ramach inspekcji pomieszczeń przez robota mobilnego.

Realizacja projektu wymagała samodzielnego połączenia metod cyfrowego przetwarzania obrazu oraz analizy geometrycznej w jeden spójny proces przetwarzania. Zastosowane rozwiązanie wykorzystuje wiedzę dotyczącą struktur i właściwości obrazów, ich przetwarzania oraz metod analizy obiektów występujących na scenie. Pozwoliło to na praktyczne wykorzystanie zagadnień omawianych w ramach przedmiotu, w szczególności segmentacji, filtracji, operacji morfologicznych, lokalizacji obiektów, transformacji Hougha oraz analizy kształtu.

Problem obejmuje analizę obrazu, na którym zegar może znajdować się zarówno w znanym, jak i nieznanym położeniu. Należy również uwzględnić przypadki, w których zegar jest tylko częściowo widoczny lub nie występuje na obrazie. W ostatniej z wymienionych sytuacji system powinien jednoznacznie zasignalizować brak zegara.

Opracowanie rozwiązania wymagało uwzględnienia zależności pomiędzy kolejnymi etapami przetwarzania oraz odpowiedniego doboru parametrów zastosowanych metod. Poszczególne operacje nie są traktowane jako niezależne procedury, lecz jako elementy jednego systemu, w którym wynik wcześniejszego etapu wpływa na skuteczność kolejnych. Takie podejście pozwala ocenić nie tylko poprawność działania pojedynczych metod, ale również ich przydatność w rzeczywistym zadaniu wizyjnym.

W ramach projektu przygotowano zbiór obrazów umożliwiający sprawdzenie działania rozwiązania dla różnych położzeń zegara i warunków obserwacji. Przeprowadzono również analizę wpływu parametrów zastosowanych algorytmów oraz warunków zewnętrznych na uzyskiwane wyniki. Na tej podstawie oceniono skuteczność opracowanego rozwiązania, wskazano jego ograniczenia oraz określono możliwości dalszego rozwoju.

Projekt stanowi tym samym praktyczną realizację zagadnień związanych z systemami wizyjnymi stosowanymi w automatyce i robotyce, wymagającą samodzielnego zaprojektowania procesu analizy, implementacji poszczególnych etapów oraz krytycznej oceny otrzymanych rezultatów.

2. Zasada działania - ogólny zarys

Przetwarzanie rozpoczyna się od wczytania pojedynczego obrazu zawierającego zegar. W pierwszym etapie model YOLO lokalizuje tarczę zegara na obrazie i zwraca jej prostokątny obszar ograniczający bounding box. Na jego podstawie z pełnego obrazu wycinany jest fragment zawierający tarczę, który następnie jest czterokrotnie powiększany z wykorzystaniem interpolacji bikubicznej. Dzięki temu kolejne etapy pracują na większej liczbie pikseli przypadających na tarczę, co ułatwia dalszą analizę jej elementów.

Na powiększonym wycinku wykonywane jest rozpoznawanie cyfr za pomocą EasyOCR. Program ogranicza zbiór rozpoznawanych znaków do cyfr, a następnie odrzuca wyniki o zbyt niskiej pewności oraz wartości spoza zakresu od 1 do 12. Dla zaakceptowanych cyfr zapisywane są również ich współrzędne na obrazie. Informacja o położeniu cyfr jest wykorzystywana w kolejnym etapie do wyznaczenia geometrii tarczy.

Środek tarczy nie jest przyjmowany jako środek wyciętego obrazu, lecz wyznaczany na podstawie rozpoznanych cyfr. Dla każdej dostępnej pary cyfr leżących naprzeciw siebie, np. 1-7, 2-8 czy 3-9, wyznaczana jest prosta przechodząca przez ich środki. Następnie obliczane są punkty przecięcia poszczególnych par prostych. Jako położenie środka tarczy przyjmowana jest mediana współrzędnych uzyskanych punktów przecięcia. Takie rozwiązanie pozwala ograniczyć wpływ pojedynczych niedokładności położenia rozpoznanych cyfr.

Po wyznaczeniu środka tworzona jest maska elementów wskazujących godzinę. Obraz tarczy jest konwertowany do skali szarości i filtrowany medianowo. Następnie w obrębie tarczy wyznaczany jest próg metodą Otsu. Na jego podstawie wybierane są obszary różniące się jasnością od tła. W celu zwiększenia odporności rozwiązania na różne warunki obrazu próg jest dodatkowo modyfikowany. Program wykorzystuje kilka wartości parametru określającego siłę progowania i dla każdej z nich wykonuje dalsze przetwarzanie.

Otrzymana maska jest oczyszczana za pomocą operacji morfologicznych. Wykorzystywane są m.in. zamknięcie i otwarcie morfologiczne, dylatacja oraz analiza spójnych obszarów. Dodatkowo z maski usuwane są obszary odpowiadające rozpoznanym cyfrom, ponieważ ich obecność mogłaby prowadzić do błędnych detekcji linii. Ostatecznie pozostawiane są przede wszystkim obszary związane ze wskazówkami i ich otoczeniem.

Na przygotowanej masce wykonywana jest probabilistyczna transformacja Hougha. Z wykrytych odcinków odrzucane są te, które są zbyt krótkie, a pozostałe są opisywane między innymi przez ich kierunek i długość względem wyznaczonego środka tarczy. Ponieważ ta sama wskazówka może zostać wykryta wielokrotnie jako kilka podobnych odcinków, zastosowano dodatkowy etap deduplikacji. Kandydaci o zbliżonym kierunku lub reprezentujący ten sam fragment wskazówki są odrzucani, dzięki czemu do dalszej analizy pozostają maksymalnie dwa reprezentatywne odcinki.

Samo określenie długości wskazówek nie opiera się jednak wyłącznie na wyniku transformacji Hougha. Dla każdego wykrytego kierunku wykonywany jest dodatkowy pomiar wzdłuż promienia wychodzącego ze środka tarczy. Program analizuje kolejne punkty wzdłuż tego promienia i sprawdza, jak długo obserwowany obszar zachowuje jasność odpowiadającą wcześniej określonymu kryterium. Zastosowanie niewielkiego obszaru wokół badanego promienia oraz tolerancji na krótkie przerwy pozwala ograniczyć wpływ pojedynczych zakłóceń obrazu. Uzyskana długość jest następnie wykorzystywana do uporządkowania wykrytych wskazówek.

Po określeniu kierunków i długości dwóch wskazówek ich kąty względem środka tarczy są przeliczane na wartości godzin i minut. Kierunek wskazówki minutowej odpowiada kolejnym sześciu stopniom na minutę, natomiast położenie wskazówki godzinowej jest przeliczane względem pełnego obrotu tarczy. W algorytmie uwzględniono również sytuację, w której położenie wskazówki minutowej znajduje się blisko pełnej godziny. Zastosowana korekta zapobiega błędnemu przypisaniu godziny, np. odczytaniu położenia odpowiadającego końcowi 12:59 jako 1:59.

Oprócz samego wyniku liczbowego program tworzy obraz wizualizujący przebieg analizy. Na obrazie zaznaczane są rozpoznane cyfry wraz z wartościami pewności OCR, linie łączące przeciwległe cyfry, wyznaczony obszar tarczy, maska wskazówek oraz wykryte wskazówki wraz z ich oznaczeniem. Pozwala to na bezpośrednią ocenę poszczególnych etapów przetwarzania i ułatwia analizę błędów.

Cały proces jest realizowany przez funkcję `read_clock`, która łączy poszczególne etapy w jeden potok: detekcję tarczy, przycięcie i skalowanie obrazu, OCR, wyznaczenie środka, segmentację wskazówek, detekcję linii, deduplikację kandydatów, dodatkowy pomiar długości oraz obliczenie czasu. Parametry poszczególnych etapów zostały zebrane w osobnym module `constants.py`, dzięki czemu ich zmiana nie wymaga modyfikowania głównej logiki algorytmu.

3. Stos technologiczny

System został podzielony na część odpowiedzialną za przetwarzanie obrazu oraz warstwę umożliwiającą wykorzystanie algorytmu za pośrednictwem przeglądarki. Logika analizy obrazu została wydzielona do modułu `clock_reader.py`, dzięki czemu może być niezależnie wykorzystywana przez aplikację przyjmującą obrazy z zewnętrznego źródła.

Za komunikację z użytkownikiem odpowiada aplikacja napisana we frameworku Flask. Po przesłaniu obrazu do endpointu `/upload` plik jest odczytywany, konwertowany do postaci obrazu OpenCV i przekazywany do funkcji odpowiedzialnej za odczyt zegara. Po zakończeniu analizy backend zwraca wynik w postaci danych JSON zawierających odczytaną godzinę oraz obraz wizualizujący przebieg analizy. Obraz wynikowy jest kodowany do formatu PNG, a następnie przesyłany w odpowiedzi jako dane Base64.

Interfejs użytkownika został wykonany z wykorzystaniem HTML, JavaScript oraz Bootstrap 5. Umożliwia przesłanie zdjęcia i prezentuje otrzymany wynik wraz z wizualizacją kolejnych elementów analizy.

Za detekcję tarczy odpowiada model YOLO uruchamiany za pośrednictwem biblioteki Ultralytics, natomiast rozpoznawanie cyfr realizowane jest przez EasyOCR. Pozostałe operacje związane z przygotowaniem obrazu, segmentacją, operacjami morfologicznymi, transformacją Hougha oraz analizą geometryczną zostały wykonane z wykorzystaniem OpenCV i NumPy.

Warstwa wdrożeniowa umożliwia uruchomienie systemu jako działającej usługi sieciowej. Gunicorn pełni rolę serwera WSGI dla aplikacji Flask, natomiast nginx działa jako reverse proxy. Za utrzymanie procesu aplikacji odpowiada Supervisor, a komunikacja z usługą może być zabezpieczona za pomocą certyfikatu Let's Encrypt i protokołu HTTPS. Repozytorium zawiera również skrypty automatyzujące konfigurację środowiska produkcyjnego.

4. Opis kodu

Projekt został podzielony na trzy główne moduły programowe. Plik `constants.py` zawiera parametry sterujące działaniem algorytmu, `app.py` odpowiada za komunikację z użytkownikiem oraz uruchomienie aplikacji webowej, natomiast `clock_reader.py` zawiera właściwy algorytm analizy obrazu i rozpoznawania czasu.

Taki podział pozwala oddzielić parametry konfiguracyjne, warstwę komunikacyjną oraz logikę przetwarzania obrazu. Jest to istotne również z punktu widzenia dalszego dostrajania rozwiązania. Zmiana parametrów progowania, tolerancji geometrycznych czy parametrów transformacji Hougha nie wymaga modyfikowania funkcji realizujących poszczególne etapy algorytmu.

4.1. `constants.py` - parametry algorytmu

Plik `constants.py` nie zawiera funkcji realizujących przetwarzanie obrazu. Jego zadaniem jest zebranie w jednym miejscu wszystkich wartości liczbowych, które wpływają na działanie algorytmu. Dzięki temu kod odpowiedzialny za przetwarzanie pozostaje czytelniejszy, a dostrajanie programu do różnych warunków obrazu może być wykonywane poprzez zmianę parametrów bez ingerowania w samą logikę funkcji.

4.1.1. Model i geometria tarczy

- `MODEL_PATH` - ścieżka do wag wytrenowanego modelu YOLO (`best.pt`), wczytywanych przez `app.py` przy starcie serwera.
- `DIAL_RADIUS_RATIO` - jaka część krótszego boku wyciętego i powiększonego obrazu tarczy ($\min(w,h)$) jest przyjmowana jako promień tarczy R . Ta wartość R jest później używana praktycznie wszędzie - przy maskowaniu wskazówek, transformacji Hougha i pomiarze promieniem.

4.1.2. OCR

- `OCR_LANGUAGES` - lista języków przekazywana do EasyOCR przy tworzeniu czytnika; wpływa na model rozpoznawania znaków, nie na sam wynik (bo i tak filtrujemy tylko cyfry).
- `OCR_GPU` - czy EasyOCR ma używać GPU. Ustawione na `False`, bo produkcyjny serwer korzysta z GPU głównie dla YOLO, a EasyOCR na CPU wystarcza.
- `OCR_MIN_CONF` - minimalna pewność odczytu, poniżej której cyfra jest odrzucana w `ocr_digits`.
- `OCR_MAG_RATIO` - współczynnik powiększenia stosowany wewnętrznie przez EasyOCR przy analizie fragmentu obrazu - im wyższy, tym drobniejsze cyfry są lepiej odczytywane, kosztem czasu.
- `OCR_ALLOWLIST` - lista dozwolonych znaków przekazywana do EasyOCR, ograniczająca rozpoznawanie tylko do cyfr 0-9.

4.1.3. Przycinanie

- `CROP_UPSCALE_FACTOR` - krotność powiększenia wyciętego fragmentu tarczy w `crop_dial` (interpolacja bikubiczna). Wartość 4 oznacza, że wycięty obszar jest skalowany czterokrotnie w obu wymiarach.

4.1.4. Maskowanie wskazówek

- DIAL_MASK_RADIUS_MARGIN - margines (mnożnik R) określający promień koła, w obrębie którego liczony jest próg Otsu i w obrębie którego w ogóle szuka się wskazówek. Wartość 1.02 oznacza margines 2% ponad wyliczony promień tarczy.
- HAND_MASK_STRENGTHS - krotka kolejnych "sił" progowania, próbowanych po kolei w pętli w read_clock, dopóki nie uda się wydzielić dwóch osobnych wskazówek. Zaczyna od najsilniejszego przesunięcia progu (1.0) i schodzi w dół do braku przesunięcia (0.0).
- HAND_MASK_MAX_THRESHOLD_SHIFT - maksymalna wartość, o jaką próg Otsu jest przesuwany (w skali jasności pikseli), skalowana przez aktualną "siłę" z HAND_MASK_STRENGTHS.
- HAND_MASK_CLOSE_KERNEL_SIZE - rozmiar jądra dla domknięcia morfologicznego, którym czyszczona jest surowa maska wskazówek zaraz po progowaniu (łączenie drobnych przerw).
- HAND_MASK_NUMBER_DILATE_KERNEL_SIZE - rozmiar jądra, którym dylatowane są prostokąty otaczające rozpoznane cyfry przed wycięciem ich z maski wskazówek - dzięki dylatacji usuwany jest też margines wokół cyfr, nie tylko sam napis.
- HAND_MASK_OPEN_KERNEL_BASE_RATIO i
HAND_MASK_OPEN_KERNEL_STRENGTH_RATIO - razem określają rozmiar jądra otwarcia morfologicznego jako ułamek promienia R: bazowy rozmiar plus dodatkowy składnik rosnący wraz z aktualną "siłą" progowania. Im silniejsze progowanie, tym większe jądro otwarcia potrzebne do usunięcia powstałego szumu.
- HAND_MASK_DILATE_ITERATIONS - liczba iteracji dylatacji wykonywanej po otwarciu, żeby odtworzyć grubość wskazówek zmniejszoną przez otwarcie.
- HAND_MASK_CENTER_RADIUS_RATIO - promień (jako ułamek R) małego koła wokół środka tarczy, używanego do wybrania tych spójnych obszarów maski, które faktycznie stykają się ze środkiem - reszta jest odrzucana jako niezwiązana ze wskazówkami.

4.1.5. Transformacja Hougha

- HOUGH_RHO - rozdzielczość odległości (w pikselach) akumulatora Hougha w HoughLinesP.
- HOUGH_THETA_DIVISOR - dzielnik π określający rozdzielczość kątową akumulatora; 360 daje krok co pół stopnia.
- HOUGH_THRESHOLD - minimalna liczba głosów w akumulatorze, żeby dany odcinek został zwrócony jako wykryty.
- HOUGH_MIN_LINE_LENGTH_RATIO - ułamek R określający minimalną długość odcinka akceptowanego przez samą funkcję HoughLinesP (parametr minLineLength).
- HOUGH_MAX_LINE_GAP_RATIO - ułamek R określający maksymalną przerwę między punktami, żeby nadal traktować je jako jeden odcinek (parametr maxLineGap).
- HOUGH_MIN_LENGTH_CROP_RATIO - dodatkowy, niezależny próg długości (tym razem jako ułamek całego kadru, nie R), stosowany już po stronie hough_candidates do odrzucenia zbyt krótkich kandydatów jako szumu.

4.1.6. Deduplikacja kandydatów

- DEDUP_SAME_DIRECTION_TOLERANCE_DEG - tolerancja kątowa (w stopniach), w obrębie której dwa kandydaty są uznawane za tę samą wskazówkę ze względu na zbliżony kierunek.
- DEDUP_SAME_ARM_TOLERANCE_DEG - tolerancja kątowa używana przy sprawdzaniu, czy kandydat jest krótszym fragmentem tego samego ramienia (linii przechodzącej przez środek w obu kierunkach, stąd sprawdzanie modulo 180 stopni).
- DEDUP_SAME_ARM_LENGTH_RATIO - próg długości względnej: kandydat jest uznawany za duplikat tego samego ramienia, jeśli jest krótszy niż ten ułamek długości już zaakceptowanego kandydata i mieści się w tolerancji kątowej ramienia.
- DEDUP_MAX_HANDS - maksymalna liczba wskazówek zatrzymywanych przez dedup_hands (dwie - godzinowa i minutowa).

4.1.7. Pomiar promieniem

- `RAY_MAX_RADIUS_RATIO` - ułamek $\min(w,h)$ określający, jak daleko od środka tarczy program szuka końca wskazówki metodą `ray_darkness_length`.
- `RAY_PATCH_RADIUS` - promień (w pikselach) małej łatki próbkowanej wokół każdego punktu na promieniu, z której liczona jest średnia jasność.
- `RAY_PERPENDICULAR_TOLERANCE` - liczba przesunięć prostopadłych do kierunku promienia sprawdzanych na każdym kroku - pozwala trafić wskazówkę nawet jeśli jest odrobinę krzywa albo szeroka.
- `RAY_GAP_TOLERANCE` - liczba kolejnych "pustych" kroków (bez trafienia w ciemny/jasny obszar) tolerowana zanim program uzna, że wskazówka się skończyła.

4.1.8. Wyznaczanie godziny

`CLOCK_HOUR_CARRY_FRACTION_LOW` i `CLOCK_HOUR_CARRY_FRACTION_HIGH` - progi ułamkowe używane w `clock_time` do decyzji o korekcie przeniesienia godziny. Porównywana jest ułamkowa część godziny (`frac`) z ułamkiem minuty (`exp`) - jeśli jedna jest blisko zera a druga blisko jedności, oznacza to że wskazówka minutowa "przeskoczyła" pełną godzinę i trzeba skorygować numer godziny w jedną lub drugą stronę.

4.1.9. Kolory i rysowanie wizualizacji

Wszystkie stałe `COLOR_*` to krotki BGR (konwencja OpenCV, nie RGB) określające kolor poszczególnych elementów rysowanych przez `draw_overlay`: `COLOR_MASK_OVERLAY` - kolor nakładki maski wskazówek, `COLOR_NUMBER_BOX` i `COLOR_NUMBER_TEXT` - kolor ramki i podpisu przy rozpoznanych cyfrach, `COLOR_PAIR_LINE` - kolor linii łączących przeciwległe cyfry użyte do wyznaczenia środka, `COLOR_DIAL_CIRCLE` - kolor okręgu oznaczającego granicę tarczy, `COLOR_MINUTE_HAND` i `COLOR_HOUR_HAND` - kolory obu wskazówek na wizualizacji, `COLOR_CENTER_DOT` - kolor kropki w wyznaczonym środku tarczy.

- `MASK_OVERLAY_ALPHA` i `MASK_BASE_ALPHA` - wagi mieszania (`addWeighted`) przy nakładaniu półprzezroczystej maski wskazówek na oryginalny obraz - pierwsza to waga nakładki, druga waga obrazu bazowego.

- `NUMBER_BOX_THICKNESS`, `NUMBER_TEXT_SCALE`, `NUMBER_TEXT_THICKNESS` - grubość ramki wokół cyfry, skala i grubość czcionki podpisu z numerem cyfry i pewnością OCR.
- `PAIR_LINE_THICKNESS` - grubość linii łączących pary przeciwległych cyfr.
- `DIAL_CIRCLE_THICKNESS` - grubość okręgu oznaczającego tarczę.
- `HAND_LINE_THICKNESS` - grubość rysowanych linii obu wskazówek.
- `HAND_LABEL_SCALE` i `HAND_LABEL_THICKNESS` - skala i grubość czcionki podpisu przy końcu każdej wskazówki.
- `CENTER_DOT_RADIUS` - promień kropki rysowanej w środku tarczy.
- `HAND_LABELS` - słownik mapujący indeks wskazówki na literowy podpis: 0 to "M" (minutowa), 1 to "G" (godzinowa) - używany przy rysowaniu etykiet w `draw_overlay`.

4.1.10. Serwer

- `FLASK_DEBUG` - przełącznik trybu debug Flaska, odczytywany w `app.py` przy uruchamianiu (`app.run(debug=c.FLASK_DEBUG)`). W środowisku produkcyjnym aplikacja jest uruchamiana przez `gunicorna`, więc ta flaga ma znaczenie głównie przy lokalnym uruchamianiu przez `python app.py`

4.2. app.py - warstwa aplikacyjna

Plik `app.py` stanowi warstwę komunikacyjną pomiędzy użytkownikiem a algorytmem przetwarzania obrazu. Nie realizuje on analizy tarczy ani detekcji wskazówek. Jego zadaniem jest odebranie

zdjęcia, przekazanie go do funkcji `read_clock`, a następnie zwrócenie wyniku w formacie umożliwiającym jego prezentację w przeglądarce.

Istotnym elementem programu jest sposób inicjalizacji YOLO i EasyOCR. Model YOLO jest ładowany instrukcją:

```
model = YOLO(c.MODEL_PATH)
```

Natomiast czytnik OCR tworzony jest przez:

```
reader = easyocr.Reader(c.OCR_LANGUAGES, gpu=c.OCR_GPU, verbose=False)
```

Operacje te wykonywane są raz podczas uruchamiania aplikacji, a nie przy każdym przesłanym zdjęciu. Ma to duże znaczenie praktyczne, ponieważ załadowanie modelu detekcyjnego oraz inicjalizacja OCR są operacjami znacznie bardziej kosztownymi niż samo przekazanie kolejnego obrazu do już utworzonych obiektów. Dzięki temu każde kolejne żądanie może korzystać z wcześniej przygotowanych modeli.

Funkcja `image_to_b64` odpowiada za przygotowanie obrazu wynikowego do przesłania w odpowiedzi HTTP. Najpierw obraz OpenCV jest kodowany do formatu PNG za pomocą `cv2.imencode()`. Jeżeli operacja zakończy się powodzeniem, otrzymany bufor bajtów jest kodowany za pomocą Base64.

Rozwiązanie to pozwala przesłać obraz bez konieczności zapisywania go jako osobnego pliku na serwerze. Frontend otrzymuje więc wynik analizy jako część odpowiedzi JSON i może bezpośrednio wykorzystać go do wyświetlania wizualizacji.

Funkcja `index()` obsługuje główną ścieżkę aplikacji /. Jej zadaniem jest wyrenderowanie pliku HTML znajdującego się w katalogu szablonów. Funkcja nie wykonuje żadnych operacji związanych z analizą obrazu - zapewnia jedynie dostęp do interfejsu użytkownika.

Najważniejszą funkcją `app.py` jest `upload()`, obsługująca żądania POST wysyłane na adres `/upload`.

W pierwszym kroku pobierany jest plik znajdujący się w polu formularza o nazwie photo. Program sprawdza, czy plik został faktycznie przesłany oraz czy posiada nazwę. Brak pliku powoduje zwrócenie odpowiedzi JSON z komunikatem błędu i kodem HTTP 400.

Następnie zawartość pliku jest odczytywana do bufora bajtów. `numpy.frombuffer()` przekształca dane do tablicy NumPy, a `cv2.imdecode()` interpretuje je jako obraz. Dzięki temu zdjęcie może zostać przekazane bezpośrednio do OpenCV, bez konieczności wcześniejszego zapisywania go na dysku. Jeżeli dekodowanie obrazu się nie powiedzie, zwracany jest odpowiedni komunikat błędu.

Po poprawnym wczytaniu obrazu wykonywane jest:

```
time, vis = read_clock(img, model, reader)
```

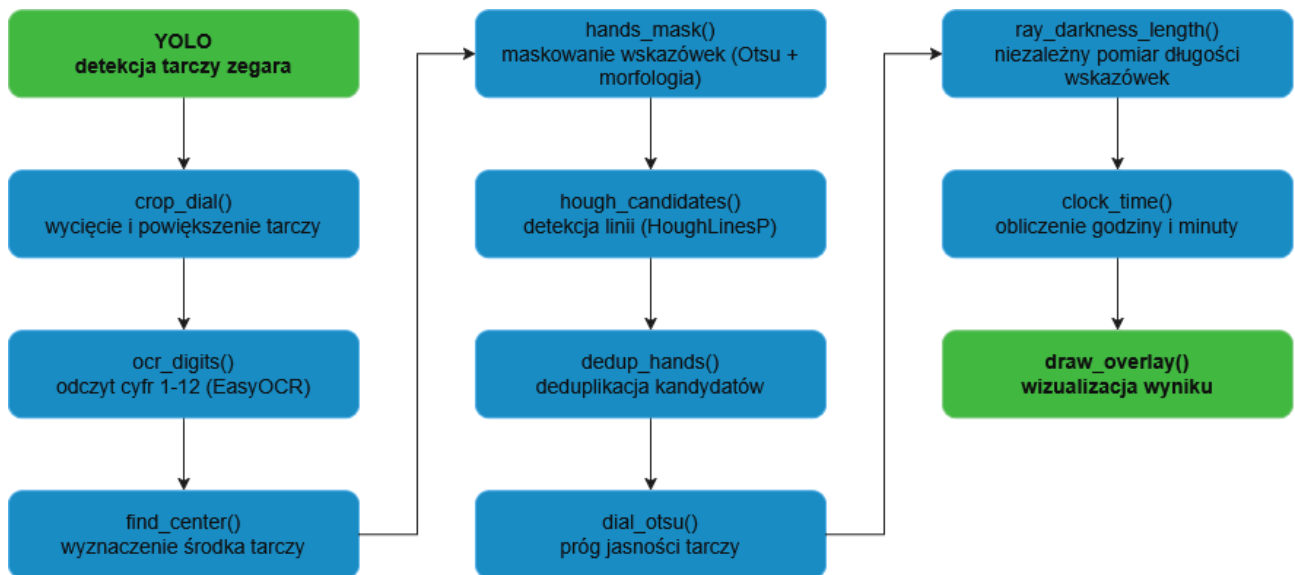
W tym miejscu warstwa aplikacyjna przekazuje sterowanie do właściwego algorytmu wizyjnego.

Jeżeli funkcja `read_clock()` nie uzyska wyniku, użytkownik otrzymuje komunikat informujący o braku zegara lub wskazówek. Jeżeli analiza zakończy się powodzeniem, godzina jest formatowana do postaci HH:MM, a obraz wizualizujący wynik zostaje zakodowany do Base64 i zwrócony w odpowiedzi JSON.

W efekcie `app.py` stanowi stosunkowo cienką warstwę pośrednią. Cała właściwa analiza obrazu pozostaje niezależna od sposobu, w jaki obraz został dostarczony do programu.

4.3. clock_reader.py - algorytm analizy obrazu

Plik `clock_reader.py` zawiera zasadniczą część projektu. Poszczególne funkcje tworzą uporządkowany potok przetwarzania, w którym wynik jednego etapu staje się danymi wejściowymi kolejnego.



Rys. 1. Uproszczony pipeline detekcji godziny.

4.3.1. crop_dial() - wycięcie i powiększenie tarczy

```
def crop_dial(img, x1, y1, x2, y2):
    return cv2.resize(img[int(y1):int(y2), int(x1):int(x2)], None,
                      fx=c.CROP_UPSCALE_FACTOR, fy=c.CROP_UPSCALE_FACTOR,
                      interpolation=cv2.INTER_CUBIC)
```

Rys. 2. Funkcja crop_dial()

Funkcja `crop_dial()` otrzymuje pełny obraz oraz cztery współrzędne prostokąta zwróconego przez YOLO. Na ich podstawie wycina fragment, następnie fragment jest powiększany za pomocą `cv2.resize()`. Do interpolacji wykorzystano `cv2.INTER_CUBIC`, czyli interpolację bikubiczną.

Powiększenie nie tworzy oczywiście nowych rzeczywistych szczegółów obrazu. Jego celem jest zwiększenie rozdzielczości roboczej fragmentu, na którym wykonywane będą kolejne operacje. Jest to szczególnie istotne dla OCR oraz późniejszej analizy geometrycznej wskazówek.

4.3.2. ocr_digits() - rozpoznawanie cyfr

Funkcja `ocr_digits()` wykorzystuje wcześniej utworzony obiekt EasyOCR do znalezienia napisów na powiększonej tarczy.

```
def ocr_digits(crop, reader, min_conf=c.OCR_MIN_CONF):
    numbers = {}
    for bbox, text, conf in reader.readtext(crop, allowlist=c.OCR_ALLOWLIST, mag_ratio=c.OCR_MAG_RATIO):
        if not text.isdigit():
            continue
        if conf < min_conf:
            continue
        val = int(text)
        if not (1 <= val <= 12):
            continue
        pts = np.array(bbox, dtype=np.float64)
        numbers[val] = {"val": val, "box": pts.astype(int),
                        "cx": pts[:, 0].mean(), "cy": pts[:, 1].mean(), "conf": conf}
    return numbers
```

Rys. 3. Funkcja ocr_digits()

Wywołanie OCR posiada dwa istotne ograniczenia. Po pierwsze allowlist jest ustawiony na ciąg "0123456789", czyli program informuje OCR, że interesują go wyłącznie cyfry. Po drugie ustawiony jest mag_ratio, który wpływa na skalowanie obrazu podczas działania OCR.

Każdy wynik OCR jest następnie poddawany filtracji. Program sprawdza, czy rozpoznany tekst składa się wyłącznie z cyfr. Następnie odrzucane są wyniki o pewności mniejszej od OCR_MIN_CONF. Aktualnie próg ten wynosi 0.75. Odrzucane są również wartości spoza zakresu od 1 do 12.

Dla zaakceptowanej cyfry zapisywane są:

- jej wartość,
- współrzędne bounding boxa,
- współrzędna środka cx,

- współrzędna środka cy,
- poziom pewności OCR.

Dzięki temu wynik OCR nie służy jedynie do odczytania samej cyfry. Informacja o jej położeniu zostaje wykorzystana później do wyznaczenia środka tarczy. Jest to ważne, ponieważ algorytm wykorzystuje OCR również jako źródło informacji geometrycznej.

4.3.3. `line_intersection()` - obliczenie przecięcia prostych

`line_intersection()` jest funkcją pomocniczą wykorzystywaną przez `find_center()`.

Jej zadaniem jest wyznaczenie punktu przecięcia dwóch prostych zdefiniowanych przez pary punktów. Dla pierwszej prostej wyznaczany jest wektor kierunkowy $d1$, a dla drugiej $d2$. Następnie obliczany jest wyznacznik. Jeżeli jego wartość jest bliska zeru, proste są równoległe lub niemal równoległe i jednoznaczny punkt przecięcia nie jest wyznaczany. W takim przypadku funkcja zwraca `None`.

W przeciwnym przypadku wyznaczany jest parametr t , a następnie współrzędne punktu przecięcia. Funkcja nie analizuje obrazu - realizuje czystą operację geometryczną. Jej wydzielenie do osobnej funkcji upraszcza implementację `find_center()`.

```
def line_intersection(p1, p2, p3, p4):
    d1 = (p2[0] - p1[0], p2[1] - p1[1])
    d2 = (p4[0] - p3[0], p4[1] - p3[1])
    det = d1[0] * d2[1] - d1[1] * d2[0]
    if abs(det) < 1e-9:
        return None
    t = ((p3[0] - p1[0]) * d2[1] - (p3[1] - p1[1]) * d2[0]) / det
    return (p1[0] + t * d1[0], p1[1] + t * d1[1])
```

Rys. 4. Funkcja `line_intersection()`

4.3.4. find_center() - wyznaczenie środka tarczy

Wyznaczenie środka tarczy jest jednym z ważniejszych elementów całego rozwiązania.

```
def find_center(numbers, w, h):
    pairs = []
    for a, b in ((1, 7), (2, 8), (3, 9), (4, 10), (5, 11), (6, 12)):
        if a in numbers and b in numbers:
            pairs.append((a, b))

    lines = [((numbers[a]["cx"], numbers[a]["cy"]), (numbers[b]["cx"], numbers[b]["cy"]))
              for a, b in pairs]

    pts = []
    for i in range(len(lines)):
        for j in range(i + 1, len(lines)):
            p = line_intersection(lines[i][0], lines[i][1], lines[j][0], lines[j][1])
            if p is not None and 0 <= p[0] < w and 0 <= p[1] < h:
                pts.append(p)

    if not pts:
        return w / 2.0, h / 2.0, pairs

    arr = np.array(pts)
    return float(np.median(arr[:, 0])), float(np.median(arr[:, 1])), pairs
```

Rys. 5. Funkcja find_center()

Zamiast przyjmować środek prostokąta zwróconego przez YOLO, funkcja find_center wyznacza środek tarczy na podstawie położenia rozpoznanych cyfr. Dla par cyfr leżących naprzeciw siebie, dla których obie cyfry zostały poprawnie odczytane przez OCR, tworzone są proste przechodzące przez ich środki, a następnie dla każdej pary takich prostych funkcja line_intersection wyznacza punkt przecięcia. Punkty leżące poza obrazem są odrzucane - pozostałe tworzą zbiór niezależnych estymacji środka.

Jeżeli nie znaleziono żadnego poprawnego przecięcia, jako rozwiązanie zastępcze przyjmowany jest środek obrazu. W przeciwnym razie środek wyznaczany jest jako mediana współrzędnych X i mediana współrzędnych Y wszystkich przecięć. Mediana zamiast średniej jest tu celowa -

pojedyncza nieznacznie błędnie zlokalizowana cyfra przesuwają jeden punkt przecięcia daleko od reszty, a mediana jest odporna na taką pojedynczą wartość odstającą, w przeciwieństwie do średniej arytmetycznej.

Funkcja zwraca trójkę: współrzędną cx , współrzędną cy oraz listę par cyfr wykorzystanych do wyznaczenia środka - ta lista jest później potrzebna wyłącznie do narysowania linii łączących pary na wizualizacji wyniku.

4.3.5. `hands_mask()` - segmentacja wskazówek

Funkcja `hands_mask()` odpowiada za przygotowanie binarnej maski, na której później wykonywana jest transformacja Hougha.

```
def hands_mask(crop, cx, cy, R, numbers, strength):
    h, w = crop.shape[:2]
    gray = cv2.medianBlur(cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY), 5)

    yy, xx = np.mgrid[0:h, 0:w]
    dist = np.hypot(xx - cx, yy - cy)
    dial = dist < c.DIAL_MASK_RADIUS_MARGIN * R

    thr, _ = cv2.threshold(gray[dial].astype(np.uint8), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    darker = np.median(gray[dial]) > thr
    shift = strength * c.HAND_MASK_MAX_THRESHOLD_SHIFT
    eff_thr = thr - shift if darker else thr + shift
    bw = ((gray < eff_thr) if darker else (gray > eff_thr)).astype(np.uint8) * 255
    bw[~dial] = 0
    bw = cv2.morphologyEx(bw, cv2.MORPH_CLOSE, np.ones(c.HAND_MASK_CLOSE_KERNEL_SIZE, np.uint8))

    num_mask = np.zeros((h, w), np.uint8)
    for n in numbers.values():
        cv2.fillPoly(num_mask, [n["box"].astype(np.int32)], 255)
    bw[cv2.dilate(num_mask, np.ones(c.HAND_MASK_NUMBER_DILATE_KERNEL_SIZE, np.uint8)) > 0] = 0

    open_k = max(3, int(round((c.HAND_MASK_OPEN_KERNEL_BASE_RATIO +
                             c.HAND_MASK_OPEN_KERNEL_STRENGTH_RATIO * strength) * R)) | 1)
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (open_k, open_k))
    bw = cv2.morphologyEx(bw, cv2.MORPH_OPEN, kernel)
    bw = cv2.dilate(bw, kernel, iterations=c.HAND_MASK_DILATE_ITERATIONS)

    n_lbl, lbl = cv2.connectedComponents(bw)
    center_lbls = set(np.unique(lbl[dist < c.HAND_MASK_CENTER_RADIUS_RATIO * R])) - {0}
    if center_lbls:
        bw = np.where(np.isin(lbl, list(center_lbls)), 255, 0).astype(np.uint8)
    return bw
```

Rys. 6. Funkcja hands_mask()

Pierwszym krokiem jest konwersja obrazu do skali szarości oraz filtracja medianowa. Filtr medianowy ogranicza wpływ drobnego szumu i pojedynczych odstających pikseli, zachowując jednocześnie stosunkowo dobrze krawędzie obiektów.

Następnie tworzony jest obszar kołowy odpowiadający tarczy. Dla każdego piksela obliczana jest odległość od wyznaczonego wcześniej środka i tylko piksele spełniające warunek związany z promieniem tarczy są uwzględniane w dalszej analizie.

W obrębie tego obszaru obliczany jest próg metodą Otsu. Następnie program sprawdza relację mediany jasności obrazu do wyznaczonego progu, aby ustalić, czy elementy poszukiwane powinny być traktowane jako obszary ciemniejsze, czy jaśniejsze od tła.

Istotnym elementem jest parametr strength. Nie jest on bezpośrednio kolejnym progiem, ale określa przesunięcie progu Otsu. Dla jednego ustawienia próg zostaje przesunięty mocniej, dla kolejnych coraz słabiej.

Dzięki temu program nie jest uzależniony od jednego, sztywnego poziomu segmentacji.

Po binaryzacji wykonywane jest domknięcie morfologiczne. Następnie tworzona jest dodatkowa maska odpowiadająca położeniu rozpoznanych cyfr. Obszary te są rozszerzane operacją dylatacji i usuwane z maski wskazówek. Ma to ograniczyć ryzyko potraktowania cyfr jako długich odcinków odpowiadających wskazówkom.

Kolejnym etapem jest otwarcie morfologiczne. Wielkość użytego jądra jest zależna zarówno od promienia tarczy, jak i od parametru strength. Następnie wykonywana jest dylatacja. Operacje te mają ograniczyć drobne elementy i połączyć fragmenty należące do większych struktur.

Na końcu wykonywane jest wyszukiwanie spójnych składowych obrazu. Zostają zachowane te komponenty, które mają piksele znajdujące się w pobliżu środka tarczy. Jest to dodatkowe

ograniczenie wynikające z geometrii zegara: prawidłowa wskazówka musi wychodzić z centralnego punktu tarczy.

4.3.6. `hough_candidates()` - wykrywanie kandydatów na wskazówki

Po przygotowaniu maski program wykonuje probabilistyczną transformację Hough.

```
def hough_candidates(bw, cx, cy, R, crop_size):
    lines = cv2.HoughLinesP(bw, c.HOUGH_RHO, np.pi / c.HOUGH_THETA_DIVISOR, threshold=c.HOUGH_THRESHOLD,
                             minLineLength=max(5, int(c.HOUGH_MIN_LINE_LENGTH_RATIO * R)),
                             maxLineGap=max(2, int(c.HOUGH_MAX_LINE_GAP_RATIO * R)))

    if lines is None:
        return []

    min_len = c.HOUGH_MIN_LENGTH_CROP_RATIO * crop_size
    out = []
    for x1, y1, x2, y2 in lines.reshape(-1, 4):
        d1, d2 = np.hypot(x1 - cx, y1 - cy), np.hypot(x2 - cx, y2 - cy)
        length, ex, ey = (d1, x1, y1) if d1 >= d2 else (d2, x2, y2)
        if length < min_len:
            continue
        angle = np.degrees(np.arctan2(ey - cy, ex - cx)) % 360.0
        out.append({"angle": angle, "length": float(length)})
    return sorted(out, key=lambda c_: -c_["length"])
```

Rys. 7. Funkcja `hough_candidates()`

Metoda ta wyszukuje odcinki prostych występujące w obrazie binarnym. W przeciwieństwie do klasycznej transformacji Hougha zwracającej parametry prostych, wersja probabilistyczna pozwala bezpośrednio uzyskać końce wykrytych odcinków.

Parametry transformacji, takie jak `threshold`, `minLineLength` oraz `maxLineGap`, są pobierane z `constants.py`. Ich wartości są zależne od wielkości analizowanej tarczy, dzięki czemu algorytm nie operuje wyłącznie na sztywnych wartościach pikselowych.

Dla każdego wykrytego odcinka obliczane są odległości jego końców od środka tarczy. Jako długość kandydata przyjmowana jest większa z tych dwóch odległości. Następnie odrzucane są odcinki, których długość nie przekracza ustalonego minimum. Dla zaakceptowanego odcinka obliczany jest również jego kąt względem środka, a następnie przeliczany ze stopniami i sprowadzany do zakresu od 0 do 360°.

Każdy kandydat jest więc reprezentowany w uproszczony sposób przez kierunek i długość.

Lista kandydatów jest następnie sortowana malejąco według długości.

4.3.7. `dedup_hands()` - usuwanie powtarzających się detekcji

Transformacja Hougha może wykryć kilka odcinków odpowiadających tej samej fizycznej wskazówce. Jest to naturalna konsekwencja pracy na masce, która może zawierać przerwy, nierówności lub kilka fragmentów jednej struktury.

```
def dedup_hands(candidates):
    accepted = []
    for cand in candidates:
        dup = False
        for a in accepted:
            same_dir = abs((cand["angle"] - a["angle"] + 180.0) % 360.0 - 180.0) <= c.DEDUP_SAME_DIRECTION_TOLERANCE_DEG
            d = abs((cand["angle"] - a["angle"]) % 180.0)
            same_arm = (cand["length"] < c.DEDUP_SAME_ARM_LENGTH_RATIO * a["length"] and
                        min(d, 180.0 - d) <= c.DEDUP_SAME_ARM_TOLERANCE_DEG)
            if same_dir or same_arm:
                dup = True
                break
        if not dup:
            accepted.append(cand)
            if len(accepted) == c.DEDUP_MAX_HANDS:
                break
    return accepted
```

Rys. 8. Funkcja `dedup_hands()`

Kandydaci są analizowani w kolejności od najdłuższego do najkrótszego. Dla każdego nowego kandydata porównywany jest jego kierunek z kierunkami wcześniej zaakceptowanych wskazówek. Jeżeli różnica kierunków mieści się w określonej tolerancji, kandydat jest traktowany jako duplikat.

Drugi warunek pozwala wykryć krótszy fragment tego samego ramienia. W tym przypadku sprawdzane są jednocześnie podobieństwo kierunku i relacja długości obu odcinków.

4.3.8. dial_otsu() - określenie charakteru wskazówki względem tła

Funkcja dial_otsu() wykonuje ponownie analizę Otsu, ale jej cel jest inny niż w przypadku hands_mask().

```
def dial_otsu(gray, cx, cy, R):
    h, w = gray.shape[:2]
    yy, xx = np.mgrid[0:h, 0:w]
    dial = np.hypot(xx - cx, yy - cy) < c.DIAL_MASK_RADIUS_MARGIN * R
    thr, _ = cv2.threshold(gray[dial].astype(np.uint8), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    darker = np.median(gray[dial]) > thr
    return thr, darker
```

Rys. 9. Funkcja dial_otsu()

Funkcja otrzymuje obraz w skali szarości oraz położenie środka i promień tarczy. Tworzona jest maska kołowa, a następnie tylko piksele znajdujące się w tym obszarze są przekazywane do cv2.threshold() z metodą Otsu.

Funkcja zwraca wyznaczony próg thr, informację darker, określającą, czy poszukiwane elementy powinny być traktowane jako ciemniejsze od tła.

4.3.9. ray_darkness_length() - niezależny pomiar długości wskazówki

Jest to jeden z ciekawszych elementów rozwiązania, ponieważ program nie ufa bezpośrednio długości odcinka wykrytego przez Hougha.

Funkcja otrzymuje kierunek wskazówki, a następnie przeprowadza analizę od środka tarczy na zewnątrz. Program nie sprawdza jednak wyłącznie pojedynczego piksela. Wokół badanego punktu analizowany jest niewielki fragment obrazu. Dodatkowo sprawdzane są punkty przesunięte w kierunku prostopadłym do promienia. Dzięki temu niewielkie odchylenie rzeczywistej wskazówki od kierunku wyznaczonego przez Hougha nie powoduje natychmiastowego zakończenia pomiaru.

Dla każdego badanego obszaru obliczana jest średnia jasność. Jeżeli spełnia ona kryterium wynikające z `dial_otsu()`, punkt jest traktowany jako należący do wskazówki.

Istotny jest również mechanizm `gap_tol`. Pojedyncze lub krótkie przerwy w obszarze wskazówki nie powodują natychmiastowego zakończenia pomiaru. Program zapamiętuje ostatni poprawny punkt i kończy analizę dopiero po przekroczeniu dopuszczalnej liczby kolejnych nieudanych próbek.

W rezultacie otrzymywana jest długość wskazówki mierzona od środka tarczy.

```

def ray_darkness_length(gray, cx, cy, angle, max_r, thr, darker,
                        patch=c.RAY_PATCH_RADIUS, perp_tol=c.RAY_PERPENDICULAR_TOLERANCE,
                        gap_tol=c.RAY_GAP_TOLERANCE):
    h, w = gray.shape[:2]
    a = np.radians(angle)
    dx, dy = np.cos(a), np.sin(a)
    ppx, ppy = -dy, dx
    last_hit, gap = 0.0, 0
    r = 0.0
    while r <= max_r:
        hit = False
        for off in range(-perp_tol, perp_tol + 1):
            x = int(round(cx + r * dx + off * ppx))
            y = int(round(cy + r * dy + off * ppy))
            x0, x1 = max(0, x - patch), min(w, x + patch + 1)
            y0, y1 = max(0, y - patch), min(h, y + patch + 1)
            if x1 > x0 and y1 > y0:
                val = gray[y0:y1, x0:x1].mean()
                if (val < thr) if darker else (val > thr):
                    hit = True
                    break
        if hit:
            last_hit, gap = r, 0
        else:
            gap += 1
            if gap > gap_tol and last_hit > 0:
                break
        r += 1.0
    return last_hit

```

Rys. 10. Funkcja `ray_darkness_length()`

4.3.10. `clock_time()` - przeliczenie geometrii na godzinę

Funkcja `clock_time()` otrzymuje listę wykrytych wskazówek. Wcześniej lista ta została uporządkowana według długości malejąco, dlatego pierwsza wskazówka jest traktowana jako minutowa, a druga jako godzinowa.

```

def clock_time(hands):
    if not hands:
        return None
    minute_hand = hands[0]
    hour_hand = hands[1] if len(hands) >= 2 else hands[0]

    minute = ((minute_hand["angle"] - 270.0) % 360.0) / 6.0 % 60.0
    hour_f = ((hour_hand["angle"] - 270.0) % 360.0) / 30.0 % 12.0
    hour = int(hour_f)

    frac, exp = hour_f - hour, minute / 60.0
    if frac < c.CLOCK_HOUR_CARRY_FRACTION_LOW and exp > c.CLOCK_HOUR_CARRY_FRACTION_HIGH:
        hour = (hour - 1) % 12
    elif frac > c.CLOCK_HOUR_CARRY_FRACTION_HIGH and exp < c.CLOCK_HOUR_CARRY_FRACTION_LOW:
        hour = (hour + 1) % 12
    hour = 12 if hour == 0 else hour
    return hour, minute

```

Rys. 11. Funkcja clock_time()

Kod nie ogranicza się jednak do prostego zaokrąglenia godziny. Wykorzystuje również część ułamkową położenia wskazówki godzinowej oraz położenie wskazówki minutowej.

Jest to potrzebne ze względu na geometrię rzeczywistego zegara. Wskazówka godzinowa przesuwa się płynnie wraz z upływem minut. Przykładowo przy 12:59 znajduje się już bardzo blisko pozycji odpowiadającej godzinie 1. Samo obcięcie części ułamkowej mogłoby więc prowadzić do błędnego przypisania godziny.

4.3.11. draw_overlay() - wizualizacja procesu

Funkcja draw_overlay() nie wpływa na wynik działania algorytmu. Jej zadaniem jest przygotowanie obrazu dokumentującego działanie poszczególnych etapów.

```

def draw_overlay(crop, cx, cy, R, numbers, pairs, bw, hands):
    vis = crop.copy()
    overlay = vis.copy()
    overlay[bw > 0] = c.COLOR_MASK_OVERLAY
    vis = cv2.addWeighted(overlay, c.MASK_OVERLAY_ALPHA, vis, c.MASK_BASE_ALPHA, 0)

    for val, n in numbers.items():
        cv2.polyline(vis, [n["box"]], True, c.COLOR_NUMBER_BOX, c.NUMBER_BOX_THICKNESS)
        bx, by = n["box"][0]
        cv2.putText(vis, f"{val} ({n['conf']:.2f})", (bx, by - 6),
                    cv2.FONT_HERSHEY_SIMPLEX, c.NUMBER_TEXT_SCALE, c.COLOR_NUMBER_TEXT, c.NUMBER_TEXT_THICKNESS)

    for a, b in pairs:
        p1 = (int(numbers[a]["cx"]), int(numbers[a]["cy"]))
        p2 = (int(numbers[b]["cx"]), int(numbers[b]["cy"]))
        cv2.line(vis, p1, p2, c.COLOR_PAIR_LINE, c.PAIR_LINE_THICKNESS)

    cv2.circle(vis, (int(cx), int(cy)), int(R), c.COLOR_DIAL_CIRCLE, c.DIAL_CIRCLE_THICKNESS)

    hand_colors = {0: c.COLOR_MINUTE_HAND, 1: c.COLOR_HOUR_HAND}
    for i, g in enumerate(hands[:2]):
        label, color = c.HAND_LABELS[i], hand_colors[i]
        a = np.radians(g["angle"])
        ex, ey = int(cx + g["length"] * np.cos(a)), int(cy + g["length"] * np.sin(a))
        cv2.line(vis, (int(cx), int(cy)), (ex, ey), color, c.HAND_LINE_THICKNESS)
        cv2.putText(vis, label, (ex + 6, ey), cv2.FONT_HERSHEY_SIMPLEX, c.HAND_LABEL_SCALE, color, c.HAND_LABEL_THICKNESS)

    cv2.circle(vis, (int(cx), int(cy)), c.CENTER_DOT_RADIUS, c.COLOR_CENTER_DOT, -1)
    return vis

```

Rys. 12. Funkcja `draw_overlay()`

Na kopię analizowanego obrazu nanoszona jest najpierw maska wskazówek. Jest ona mieszana z obrazem bazowym przy wykorzystaniu `cv2.addWeighted()`, dzięki czemu użytkownik może jednocześnie zobaczyć oryginalną tarczę oraz obszary zakwalifikowane przez segmentację.

Następnie dla każdej rozpoznanej cyfry rysowany jest jej bounding box oraz tekst zawierający rozpoznaną wartość i poziom pewności OCR.

Kolejnym elementem są linie łączące przeciwległe cyfry. Są one szczególnie przydatne podczas dokumentowania działania funkcji `find_center()`, ponieważ pokazują bezpośrednio, z jakich danych geometrycznych wyznaczono środek tarczy.

Na obrazie rysowany jest również okrąg odpowiadający przyjętemu obszarowi tarczy oraz punkt oznaczający jej środek.

Na końcu rysowane są obie wykryte wskazówki.

Tak przygotowany obraz jest szczególnie istotny w kontekście dokumentacji projektu. Pozwala zweryfikować, czy błędny wynik wynikał np. z niepoprawnego OCR, błędnego wyznaczenia środka, segmentacji czy detekcji linii.

4.4. read_clock() - główny potok przetwarzania

Funkcja `read_clock()` jest funkcją nadrzędną całego modułu. Nie implementuje pojedynczej metody przetwarzania obrazu, lecz koordynuje wszystkie wcześniejsze funkcje w jeden kompletny algorytm.

Pierwszym etapem jest wywołanie modelu YOLO. Model otrzymuje cały obraz i zwraca wykryte obiekty należące do klasy odpowiadającej tarczy zegara. Jeżeli obiekt zostanie znaleziony, pobierane są współrzędne pierwszego bounding boxa. Na ich podstawie `crop_dial()` wycina tarczę i powiększa ją czterokrotnie.

Następnie uruchamiany jest OCR. Wyniki OCR są następnie przekazywane do `find_center()`, który wyznacza środek tarczy na podstawie geometrii cyfr. Po określeniu środka ustalany jest promień roboczy tarczy.

Następnie rozpoczyna się właściwa próba segmentacji wskazówek. Program iteruje po wartościach `HAND_MASK_STRENGTHS`. Dla każdej wartości wykonywane są kolejno:

- `hands_mask()` - przygotowanie maski,
- `hough_candidates()` - detekcja odcinków,
- `dedup_hands()` - usunięcie powtarzających się kandydatów.

Jeżeli po którymś ustawieniu uda się uzyskać co najmniej dwie wskazówki, pętla zostaje przerwana. To rozwiązanie jest istotnym elementem odporności algorytmu. Zamiast zakładać, że jeden próg segmentacji będzie działał dla każdego zdjęcia, program sam próbuje kilku wariantów i wykorzystuje wynik kolejnych etapów jako kryterium powodzenia.

Po zakończeniu detekcji wykonywany jest dodatkowy pomiar długości wskazówek. Najpierw `dial_otsu()` określa próg i kierunek kontrastu, a następnie dla każdego kandydata wywoływana jest `ray_darkness_length()`. Uzyskana długość zastępuje wcześniejszą długość wynikającą z detekcji Hougha.

Następnie wskazówki są sortowane według zmierzonej długości. Dzięki temu dłuższa wskazówka trafia na pierwsze miejsce listy i jest interpretowana przez `clock_time()` jako wskazówka minutowa, a krótsza jako godzinowa.

Ostatnie dwa etapy to obliczenie czasu oraz przygotowanie wizualizacji.

4.5. Przepływ danych w programie

Cały algorytm można więc przedstawić jako ciąg zależności, w którym każdy etap dostarcza informacji wykorzystywanych przez następny. Tak zaprojektowany przepływ pokazuje, że poszczególne metody nie są zastosowane przypadkowo.

Detekcja YOLO ogranicza obszar zainteresowania, OCR dostarcza zarówno informacji semantycznej, jak i geometrycznej, geometria pozwala wyznaczyć środek tarczy, segmentacja

przygotowuje dane dla Hougha, Hough określa kierunki kandydatów, a niezależny pomiar promieniowy pozwala określić długości wskazówek i rozróżnić ich funkcję. Dopiero po połączeniu tych informacji możliwe jest obliczenie czasu.

To właśnie połączenie kilku metod klasycznego przetwarzania obrazu z detekcją obiektu i OCR stanowi zasadniczą część opracowanego rozwiązania.

5. Uczenie modelu YOLO

Jednym z kluczowych etapów realizacji projektu było przygotowanie własnego detektora odpowiedzialnego za lokalizację tarczy zegara na obrazie. Zastosowanie gotowego modelu detekcyjnego bez dodatkowego dostosowania do badanego problemu nie gwarantowałoby uzyskania stabilnych wyników, szczególnie w przypadku zdjęć wykonywanych w warunkach innych niż typowe obrazy znajdujące się w zbiorach treningowych. Z tego względu model YOLO został dostosowany do konkretnego zadania poprzez przeprowadzenie własnego procesu uczenia.

5.1. Przygotowanie zbioru danych

Jako źródło obrazów wykorzystano bazę ImageNet. Dostęp do zbioru uzyskano w ramach statusu Researcher, a do przygotowania danych wybrano obrazy należące do kategorii n02708093, odpowiadającej zegarom analogowym.

Sam wybór zdjęć był pierwszym etapem wymagającym odpowiedniego przygotowania danych. Nie chodziło jedynie o zgromadzenie możliwie dużej liczby obrazów zegarów, ale przede wszystkim o zapewnienie odpowiedniej różnorodności przykładów. Model docelowo miał być wykorzystywany

do analizy zdjęć wykonanych w warunkach rzeczywistych, dlatego zbiór powinien obejmować różne sposoby przedstawienia tego samego obiektu.

Uwzględniono między innymi zdjęcia wykonywane z różnej odległości, pod różnym kątem oraz przy odmiennym oświetleniu. Istotne były również różnice w wyglądzie samych zegarów, ich tarcz, kolorystyki oraz tła. Szczególnie problematyczne okazały się obrazy zawierające odbicia światła, częściowe zacienienie tarczy oraz perspektywiczne zniekształcenie wynikające z obserwacji zegara pod kątem.

Tak przygotowany zbiór miał pełnić nie tylko funkcję materiału treningowego, ale również możliwie dobrze odzwierciedlać warunki, z którymi detektor będzie miał do czynienia podczas późniejszego działania całego systemu.

5.2. Ręczna anotacja obrazów

Obrazy wykorzystane do treningu nie zawierały gotowych adnotacji odpowiadających dokładnie wymaganiom projektowanego detektora. Konieczne było więc samodzielne przygotowanie danych uczących.

Do anotacji wykorzystano aplikację Label Studio. Na każdym wybranym obrazie ręcznie zaznaczano prostokątny obszar obejmujący tarczę zegara. W ten sposób dla każdego przykładu utworzono informację o położeniu obiektu, którą następnie można było wykorzystać podczas uczenia modelu detekcyjnego.

Anotacja bounding boxów wymagała zachowania możliwie dużej spójności pomiędzy obrazami. Prostokąt powinien obejmować tarczę zegara, ale jednocześnie nie powinien zawierać niepotrzebnie dużych fragmentów otoczenia. Ma to znaczenie, ponieważ podczas treningu model uczy się zależności pomiędzy wyglądem obiektu a zaznaczonym obszarem. Niespójne lub niedokładne oznaczenia mogłyby więc pogorszyć jakość detekcji.

Na tym etapie istotna była również kontrola przygotowanych danych. Błędy anotacji, nieprawidłowo zaznaczone obiekty czy przypadki, w których zegar był zbyt mało widoczny, mogły bezpośrednio przełożyć się na błędy modelu.

5.3. Proces treningu

Uczenie modelu przeprowadzono w środowisku Google Colab, wykorzystując kartę graficzną NVIDIA T4. Zastosowanie GPU było istotne ze względu na charakter treningu sieci neuronowej i konieczność wielokrotnego przetwarzania całego zbioru obrazów.

Proces nie ograniczył się do pojedynczego uruchomienia treningu. Po pierwszych próbach konieczne było sprawdzenie, jak wytrenowany model zachowuje się na obrazach odpowiadających rzeczywistemu zastosowaniu. Szczególne znaczenie miały przypadki, które później miały pojawiać

się na wejściu całego systemu, a więc zdjęcia wykonane telefonem, z różnej perspektywy i w niekontrolowanych warunkach oświetleniowych.

Wyniki kolejnych treningów pozwoliły zidentyfikować sytuacje, w których model miał problemy z prawidłowym wskazaniem tarczy. Samo zwiększanie liczby obrazów nie było wystarczającym rozwiązaniem. Istotniejsza okazała się różnorodność danych oraz obecność w zbiorze przykładów odpowiadających trudnym przypadkom testowym.

Z tego względu proces przygotowania detektora miał charakter iteracyjny. Po kolejnych treningach analizowano uzyskane wyniki, identyfikowano przypadki problematyczne, a następnie odpowiednio modyfikowano zbiór danych. Obejmowało to dobór kolejnych obrazów oraz ich ponowną anotację. Zaktualizowany zbiór był następnie ponownie wykorzystywany do treningu modelu.

6. Wdrożenie produkcyjne

Po zakończeniu prac nad algorytmem odczytu czasu konieczne było przygotowanie aplikacji do uruchomienia w środowisku serwerowym. Samo uruchomienie programu w środowisku deweloperskim nie było wystarczające, ponieważ docelowo system miał działać jako aplikacja webowa, do której użytkownik może przelać zdjęcie zegara i otrzymać wynik przetwarzania. Z tego względu opracowano sposób wdrażania obejmujący zarówno przygotowanie środowiska systemowego, jak i konfigurację procesu aplikacji oraz serwera WWW.

W repozytorium znajdują się dwa skrypty odpowiedzialne za kolejne etapy wdrożenia `install_systemywizyjne.sh` oraz `secure_systemywizyjne.sh`. Rozdzielenie tych czynności pozwala najpierw przygotować działające środowisko aplikacji, a następnie skonfigurować jego dostęp za pomocą domeny i certyfikatu SSL.

6.1. Automatyzacja instalacji środowiska

Pierwszym etapem wdrożenia jest uruchomienie skryptu `install_systemywizyjne.sh`. Jego zadaniem jest przygotowanie serwera w taki sposób, aby po wykonaniu skryptu aplikacja posiadała wszystkie podstawowe elementy wymagane do działania.

Na początku skrypt aktualizuje listę pakietów systemowych oraz wykonuje aktualizację zainstalowanych pakietów. Następnie instalowane są podstawowe narzędzia i komponenty środowiska: git, Python wraz z modułem venv, pip, narzędzia deweloperskie, nginx oraz supervisor. Dzięki temu nie jest konieczne ręczne przygotowywanie każdego elementu serwera przed uruchomieniem aplikacji.

Kolejnym krokiem jest pobranie repozytorium projektu do katalogu `/home/systemywizyjne`. Następnie tworzona jest wirtualna instalacja Pythona w katalogu `.venv`. Pozwala to odseparować biblioteki wymagane przez projekt od pozostałych pakietów Pythona znajdujących się w systemie. Zależności aplikacji są instalowane na podstawie pliku `requirements.txt`.

Istotnym elementem skryptu jest również konfiguracja uprawnień. Katalog projektu otrzymuje właściciela `www-data`, ponieważ właśnie jako ten użytkownik uruchamiany jest proces Gunicorna. Jednocześnie użytkownik wykonujący wdrożenie otrzymuje dostęp grupowy do plików projektu. Takie rozwiązanie pozwala zachować możliwość administracji środowiskiem bez uruchamiania każdej operacji z uprawnieniami administratora.

6.2. Uruchamianie aplikacji za pomocą Gunicorn i Supervisor

W środowisku produkcyjnym aplikacja Flask nie jest uruchamiana za pomocą wbudowanego serwera deweloperskiego. Skrypt tworzy konfigurację supervisor, która odpowiada za uruchomienie aplikacji poprzez Gunicorn.

Gunicorn jest uruchamiany z wykorzystaniem środowiska wirtualnego projektu i wskazuje obiekt app znajdujący się w pliku app.py. Proces jest uruchamiany jako użytkownik www-data, a Supervisor otrzymuje również konfigurację automatycznego uruchamiania i ponownego uruchamiania procesu. W przypadku zakończenia procesu aplikacji mechanizm nadzorujący może więc uruchomić go ponownie.

W konfiguracji zastosowano jeden worker: `gunicorn -w 1 -b 0.0.0.0:80 app`

Jest to istotne ograniczenie aktualnej wersji systemu. Jeden worker oznacza, że przetwarzanie kolejnych żądań przez warstwę aplikacji nie jest skalowane na wiele równoległych procesów wykonujących analizę obrazu. Rozwiązanie to było wystarczające dla przyjętego sposobu użytkowania aplikacji, ale przy większej liczbie jednoczesnych użytkowników wymagałoby dalszej rozbudowy architektury.

Supervisor zapisuje również standardowe wyjście oraz komunikaty błędów aplikacji do osobnych plików logów. Ułatwia to późniejszą diagnostykę problemów występujących już po wdrożeniu.

6.3. Konfiguracja Nginx

Drugim elementem warstwy serwerowej jest Nginx. W projekcie pełni on funkcję reverse proxy, czyli pośredniczy pomiędzy użytkownikiem a aplikacją uruchomioną przez Gunicorna.

Skrypt instalacyjny tworzy konfigurację serwera Nginx, w której żądania przychodzące do serwera są przekazywane do aplikacji. Jednocześnie przekazywane są podstawowe informacje dotyczące żądania, między innymi adres klienta oraz użyty protokół. Konfiguracja jest następnie testowana

poleceniem `nginx -t`, a usługa Nginx zostaje włączona do automatycznego uruchamiania wraz z systemem.

Takie rozdzielenie odpowiedzialności pozwala oddzielić właściwą aplikację Flask od warstwy obsługi ruchu HTTP. Flask wraz z Gunicornem zajmuje się logiką aplikacji i przetwarzaniem obrazu, natomiast Nginx obsługuje warstwę serwera WWW i przekazuje żądania do backendu.

6.4. Konfiguracja HTTPS

Drugim skryptem jest `secure_systemy wizyjne.sh`. Jego zadaniem jest przygotowanie konfiguracji domenowej oraz uzyskanie certyfikatu SSL za pomocą Let's Encrypt.

Skrypt modyfikuje wcześniej utworzoną konfigurację Nginx, zastępując domyślną nazwę serwera nazwą domenową projektu. Następnie ponownie sprawdzana jest poprawność konfiguracji Nginx. Instalowany jest Certbot wraz z rozszerzeniem przeznaczonym do współpracy z Nginx, po czym wykonywane jest polecenie uzyskania i skonfigurowania certyfikatu dla wskazanej domeny.

Dodatkowo skrypt aktywuje `certbot.timer`, dzięki czemu mechanizm automatycznego odnawiania certyfikatu może działać jako usługa systemowa. Dzięki temu certyfikat nie wymaga ręcznego odnawiania przy każdym zbliżającym się terminie wygaśnięcia.

W rezultacie otrzymano kompletny łańcuch obsługi żądania: użytkownik korzysta z przeglądarki, żądanie trafia do Nginx, następnie przekazywane jest do aplikacji uruchomionej przez Gunicorn, a właściwe przetwarzanie obrazu wykonywane jest przez funkcję `read_clock()`.

6.5. Dobór zasobów serwera

Istotnym elementem wdrożenia był również dobór odpowiedniego środowiska sprzętowego. Wymagania aplikacji okazały się większe niż początkowo zakładano. Wynikało to przede wszystkim z konieczności jednoczesnego załadowania modelu YOLO, biblioteki EasyOCR oraz komponentów odpowiedzialnych za przetwarzanie obrazów.

Pierwsza próba uruchomienia aplikacji została wykonana na serwerze wyposażonym w 4 GB pamięci RAM. Podczas działania aplikacji występowały problemy wynikające z niewystarczającej ilości dostępnej pamięci. Kolejna konfiguracja, dysponująca 8 GB RAM, również nie zapewniała oczekiwanej stabilności działania.

Dopiero zastosowanie serwera wyposażonego w układ GPU oraz 16 GB pamięci RAM pozwoliło uzyskać stabilne środowisko pracy aplikacji. Problem ten pokazał, że wymagania sprzętowe systemu wizyjnego nie wynikają wyłącznie z rozmiaru samego kodu aplikacji. Znaczną część zasobów zajmują biblioteki uczenia maszynowego i przetwarzania obrazu oraz załadowane modele. Dobór serwera był więc również elementem procesu projektowego. Wymagał sprawdzenia rzeczywistego zachowania aplikacji, obserwacji błędów oraz kolejnego dostosowania środowiska do wymagań całego łańcucha przetwarzania.

6.6. Ograniczenia konfiguracji produkcyjnej

Aktualna konfiguracja została przygotowana przede wszystkim z myślą o poprawnym i stabilnym działaniu pojedynczej instancji aplikacji. Najważniejszym ograniczeniem jest wykorzystanie jednego workera Gunicorna. Oznacza to, że rozwiązanie nie zostało przygotowane do skalowania poprzez równoległe uruchamianie wielu procesów wykonujących analizę obrazów.

Przy większym obciążeniu konieczne byłoby rozważenie zwiększenia liczby workerów lub zastosowania innego mechanizmu skalowania. Wymagałoby to jednak uwzględnienia znacznego zużycia pamięci przez model YOLO, EasyOCR i pozostałe biblioteki. Zwiększenie liczby procesów

nie jest więc prostą zmianą konfiguracji, ponieważ każdy dodatkowy proces może wymagać własnych zasobów pamięci.

Zastosowana konfiguracja stanowi kompromis pomiędzy prostotą wdrożenia, stabilnością działania oraz wymaganiami sprzętowymi systemu.

Pomimo uzyskania działającego systemu, podczas realizacji projektu zidentyfikowano kilka ograniczeń wynikających zarówno z przyjętej architektury, jak i z zakresu projektu.

6.7. Pasek postępu w interfejsie użytkownika

Jednym z elementów interfejsu aplikacji jest pasek postępu wyświetlany podczas przetwarzania przesłanego zdjęcia. Jego działanie nie jest jednak bezpośrednio powiązane z rzeczywistym postępem algorytmu wykonywanego po stronie serwera.

Aktualna implementacja interfejsu wykorzystuje przygotowaną sekwencję wartości procentowych odtwarzaną po stronie przeglądarki. Oznacza to, że prezentowany użytkownikowi postęp ma charakter wizualny i nie reprezentuje dokładnie zakończenia kolejnych etapów funkcji `read_clock()`.

Rzeczywiste raportowanie postępu wymagałoby wprowadzenia dodatkowego mechanizmu komunikacji pomiędzy backendem i frontendem. Serwer musiałby zapisywać aktualny etap przetwarzania konkretnego żądania, natomiast przeglądarka musiałaby okresowo pobierać tę informację albo otrzymywać ją w sposób asynchroniczny.

W praktyce można byłoby w tym celu zastosować dodatkową warstwę przechowywania stanu, np. Redis, oraz mechanizm identyfikowania poszczególnych zadań. W ramach projektu rozwiązanie takie nie zostało wprowadzone, ponieważ zwiększałoby złożoność aplikacji, nie wpływając bezpośrednio na jakość właściwego algorytmu rozpoznawania zegara.

6.8. Ograniczona równoległość przetwarzania

Kolejnym ograniczeniem jest sposób uruchomienia Gunicorna. Aktualna konfiguracja wykorzystuje jednego workera, co ogranicza możliwość równoległego przetwarzania wielu żądań.

Ograniczenie to jest szczególnie istotne dla systemu wizyjnego, ponieważ analiza pojedynczego zdjęcia obejmuje uruchomienie kilku wymagających obliczeniowo etapów, w tym detekcji YOLO, OCR oraz szeregu operacji OpenCV. Zwiększenie liczby workerów mogłoby poprawić obsługę wielu użytkowników, ale jednocześnie zwiększyłoby zapotrzebowanie na pamięć i zasoby obliczeniowe.

W związku z tym obecna konfiguracja została dobrana jako rozwiązanie wystarczające dla założonego sposobu wykorzystania systemu, a nie jako architektura przeznaczona do obsługi dużego ruchu.

6.9. Zależność działania od jakości zdjęcia

Istotnym ograniczeniem pozostaje również jakość obrazu wejściowego. Cały proces analizy jest wieloetapowy, dlatego błąd na jednym z wcześniejszych etapów może wpływać na kolejne etapy przetwarzania. Niepoprawne wykrycie tarczy może spowodować błędne przycięcie obrazu, co z kolei utrudni rozpoznanie cyfr przez OCR, wyznaczenie środka tarczy oraz późniejszą detekcję wskazówek.

Podobnie problemy z kontrastem, odbiciami światła, zasłonięciem części tarczy lub niewystarczającą widocznością wskazówek mogą wpływać na wynik segmentacji i detekcji linii.

Ograniczenie to jest naturalną konsekwencją zastosowania klasycznych metod przetwarzania obrazu w kolejnych etapach pipeline'u. Zastosowane parametry zostały dobrane tak, aby rozwiązanie było możliwie odporne na różne warunki, jednak nie eliminuje to wpływu jakości obrazu wejściowego na końcowy wynik.

6.10. Zakres dalszej rozbudowy

Opracowana wersja systemu stanowi kompletną realizację przyjętego zadania, jednak jej architektura pozostawia możliwość dalszego rozwoju. Możliwe kierunki obejmują między innymi

rzeczywiste raportowanie postępu, zwiększenie możliwości równoległej obsługi użytkowników, dalszą optymalizację zużycia pamięci oraz rozszerzenie zbioru treningowego modelu YOLO o kolejne przypadki i warunki obrazowania.

Dalsze prace mogłyby również obejmować bardziej rozbudowaną diagnostykę błędów poszczególnych etapów pipeline'u. Pozwoliłoby to nie tylko zwracać użytkownikowi informację o nieudanym odczycie, ale również określać, czy problem wystąpił podczas detekcji tarczy, rozpoznawania cyfr, segmentacji wskazówek czy analizy geometrycznej.

7. Przykładowe wyniki

Zaprezentowane obrazy pokazują kolejne etapy analizy oraz końcowy wynik rozpoznawania czasu dla różnych przypadków położenia i widoczności zegara. Wyniki pozwalają również ocenić poprawność działania zastosowanych metod przetwarzania obrazu.



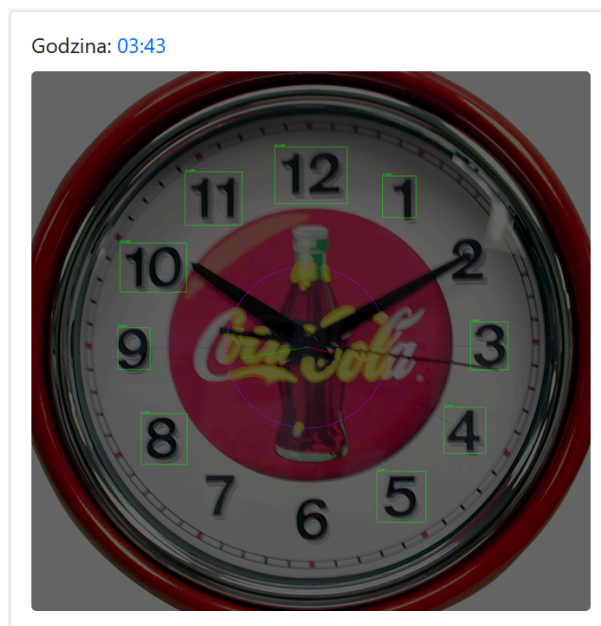
Rys. 13. Zdjęcie wsadowe



Rys. 14. Zdjęcie wyjściowe (na zielono zaznaczono odnalezione cyfry, na żółto maska, na fioletowo ograniczenie tworzenia maski)



Rys. 15. Zdjęcie wsadowe



Rys. 16. Zdjęcie wyjściowe - przez niejednorolite tło program nie zadziałał poprawnie



Rys. 17. Zdjęcie wsadowe (inna godzina niż typowa 10:10)



Rys. 18. Zdjęcie wyjściowe - poprawnie odczytana godzina mimo godziny innej do 10:10 oraz dodatkowych liczb na tarczy.

8. Podsumowanie

W ramach projektu opracowano kompletny system wizyjny przeznaczony do automatycznego odczytu czasu z analogowego zegara na podstawie obrazu. Rozwiązanie obejmuje cały łańcuch przetwarzania - od lokalizacji tarczy zegara na obrazie, poprzez przygotowanie i analizę jej fragmentu, aż do wyznaczenia położenia wskazówek i przeliczenia ich kierunku na konkretną godzinę.

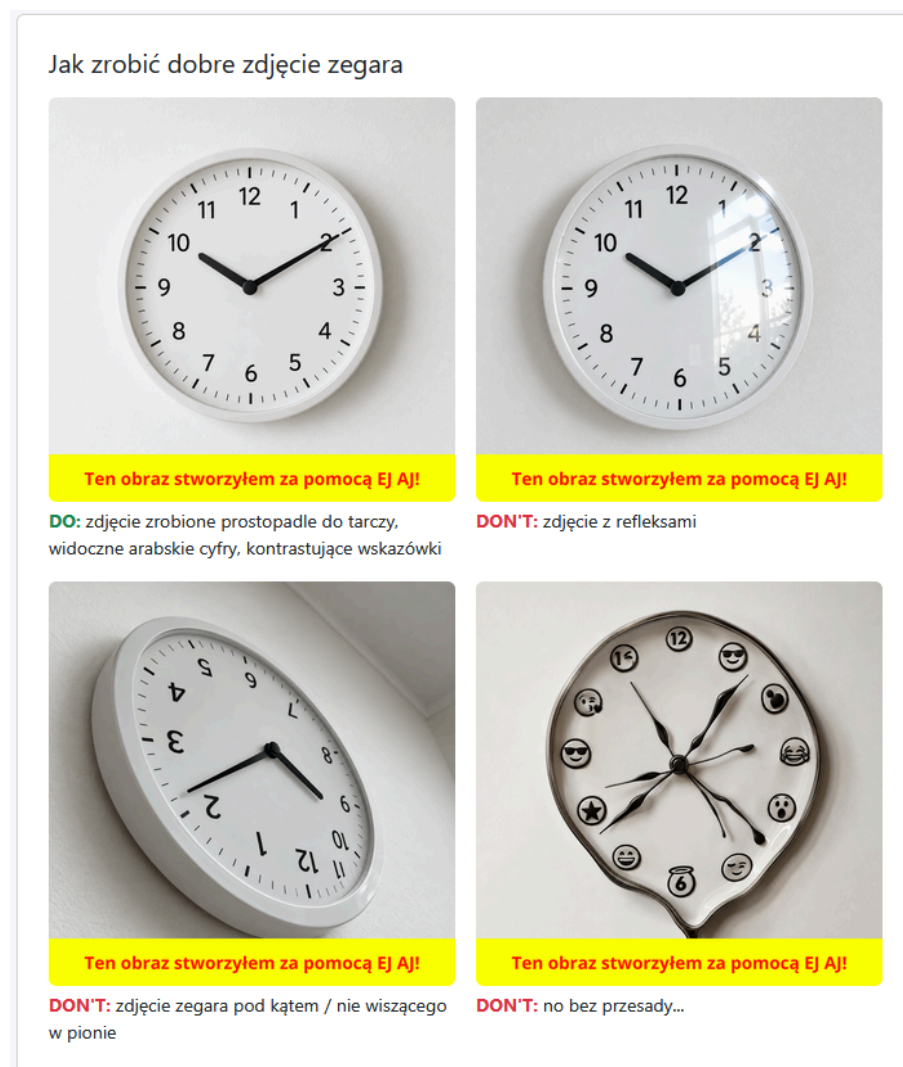
Zastosowany algorytm łączy kilka różnych metod przetwarzania obrazu. Po wykryciu tarczy przez wytrenowany model YOLO obraz jest odpowiednio przycinany i powiększany, a następnie wykonywane jest rozpoznawanie cyfr za pomocą EasyOCR. Na podstawie położeń rozpoznanych cyfr wyznaczany jest geometryczny środek tarczy. Informacja ta jest następnie wykorzystywana w dalszej analizie obrazu.

Kolejny etap obejmuje segmentację wskazówek z wykorzystaniem progowania Otsu, operacji morfologicznych oraz iteracyjnej zmiany parametrów maskowania. Na przygotowanej masce wyszukiwane są kandydackie odcinki za pomocą probabilistycznej transformacji Hougha. Wyniki są następnie filtrowane w celu ograniczenia liczby powtarzających się detekcji. Dodatkowy pomiar zasięgu wskazówek wykonywany jest metodą analizy ciemności wzdłuż promienia wychodzącego ze środka tarczy. Pozwala to określić, która z wykrytych wskazówek jest wskazówką minutową, a która godzinową.

Istotną częścią projektu było również przygotowanie własnego modelu detekcji tarczy zegara. Model YOLO został dostosowany do konkretnego zadania, a następnie zintegrowany z pozostałymi elementami systemu. Dzięki temu detekcja obiektu nie jest niezależnym eksperymentem, lecz stanowi pierwszy etap całego procesu analizy obrazu. Błąd lokalizacji tarczy może bowiem bezpośrednio wpływać na jakość wszystkich kolejnych operacji.

Projekt obejmował także przygotowanie aplikacji webowej umożliwiającej przesłanie zdjęcia oraz prezentację uzyskanego wyniku. Backend został zrealizowany w technologii Flask, natomiast wynik przetwarzania zwracany jest w postaci danych JSON zawierających rozpoznany czas oraz obraz z naniesioną wizualizacją działania algorytmu.

Końcowym etapem prac było wdrożenie aplikacji w środowisku produkcyjnym. Przygotowano automatyzację instalacji środowiska, konfigurację Gunicorna i Supervisora, serwera Nginx oraz certyfikatu SSL obsługiwane przez Let's Encrypt i Certbota. Pozwoliło to przejść od rozwiązania uruchamianego lokalnie do kompletnej usługi dostępnej przez przeglądarkę.



Rys. 19. Wymagania dotyczące wrzucanych zdjęć.

Realizacja projektu wymagała więc nie tylko implementacji pojedynczego algorytmu rozpoznawania obrazu, ale zaprojektowania i zintegrowania kilku kolejnych etapów przetwarzania. Szczególnie istotne okazało się uwzględnienie zależności pomiędzy tymi etapami, dobór parametrów metod do rzeczywistych obrazów oraz sprawdzenie zachowania rozwiązania w środowisku produkcyjnym. W efekcie powstał kompletny system łączący metody cyfrowego przetwarzania obrazu, detekcję obiektów, OCR, analizę geometryczną oraz elementy wymagane do uruchomienia aplikacji jako działającej usługi.